Number Systems, Representation, and Arithmetic Hardware

Week 2 — What the Bits Mean, and How Hardware Adds Them

Dr. Auqib Hamid Lone
Assistant Professor in Computer Science & Engineering

PCC CS-401: Computer Organization and Architecture

Department of Computer Science & Engineering
Government College of Engineering & Technology (GCET), Ganderbal

August 10, 2026

Where We Left Off (Week 1)

Week 1 was about *speed*

- Architecture (the interface) vs. organization (the implementation).
- The performance equation: CPU time $=$ IC $\times$ CPI$/f$ — three independent levers.
- Amdahl's Law: the unimproved fraction sets a hard ceiling.

The pivot today

The equation assumed we can *count* instructions and *measure* CPI. But the CPU cannot execute anything until the **data** it works on is represented in bits — and arithmetic is implemented in hardware. This week: what the bits mean, and how hardware adds.

Roadmap: Four Questions, One Thread

1. **Number systems** — how do we write numbers in binary, octal, hex — and convert?
2. **Signed representation** — how does hardware represent negative integers (two's complement)?
3. **Overflow and fixed point** — when does arithmetic fail, and how are fractions represented?
4. **Adder hardware** — how do we build a circuit that actually adds?

Running thread for today

We will add $+7 + (-3)$ in binary — bit by bit — then build the circuit that does it, and check the result against the range of the format.

The bits are meaningless until we agree on what they mean.

Motivation: One Value, Many Alphabets

Socratic question

The decimal number 13 — how would you write it if you had only two symbols, or eight, or sixteen? Each is the *same* value, different alphabet.

- Decimal $13 = 1 \times 10^1 + 3 \times 10^0$.
- Binary: $1101_2 = 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 13$.
- Octal 15_8, hex D_{16} — each a shorthand for groups of bits.
- Hardware stores *bits*; the base is just how we write them down.

Definition: Positional Number Systems

Positional notation

In base r, a number with digits $d_{n-1} \dots d_1 d_0$ has value

$$d_{n-1} r^{n-1} + \cdots + d_1 r^1 + d_0 r^0.$$

The base is written as a subscript: 1101_2, 15_8, D_{16}.

- $r = 2$ binary (digits 0,1), $r = 8$ octal (0–7), $r = 10$ decimal, $r = 16$ hex (0–9, A–F).
- **Conversion**: repeated division by r gives the digits from least to most significant.
- Hex/octal are just binary in **groups**: 1 hex digit $= 4$ bits, 1 octal digit $= 3$ bits.

Worked Example: Decimal to Binary, and Back

$53_{10} \to$ binary (repeated division by 2)

$53/2 = 26$ r 1; $26/2 = 13$ r 0; $13/2 = 6$ r 1; $6/2 = 3$ r 0; $3/2 = 1$ r 1; $1/2 = 0$ r 1.
Read remainders bottom-up: 110101_2.

Check: $110101_2 \to$ decimal

$1{\times}32 + 1{\times}16 + 0{\times}8 + 1{\times}4 + 0{\times}2 + 1{\times}1 = 32 + 16 + 4 + 1 = 53$. Correct.

Binary $\leftrightarrow$ hex is trivial by grouping: $1101\,0101_2 = D5_{16}$ — no arithmetic needed, just 4-bit groups.

Socratic Check: How Many Bits?

Question

How many bits do you need to represent n distinct values? And with k bits, what is the largest unsigned integer?

- n values $\Rightarrow \lceil \log_2 n \rceil$ bits.
- k bits $\Rightarrow$ values 0 to $2^k - 1$ (unsigned).
- Example: 8 bits $\Rightarrow 0$ to 255; 16 bits $\Rightarrow 0$ to 65535.
- This ceiling will matter the moment we talk about **overflow**.

Unsigned is easy. The hard question: how do we represent negative numbers?

Three Candidate Schemes for Negatives

We have n bits. How do we encode a sign?

- Sign-magnitude: one sign bit + $n-1$ magnitude bits. Simple, but two zeros ($+0$ and -0) and awkward addition.
- One's complement: negate = flip all bits. Still two zeros; addition needs end-around carry.
- Two's complement: negate = flip all bits and add 1. One zero, and *addition just works*.

Why two's complement wins

$-x$ is defined so that $x + (-x) = 0$ *in the same bit width*, with the carry out simply discarded. The ALU needs only an adder.

Definition: Two's Complement

Two's complement of an n-bit number

$$-x \equiv 2^n - x \pmod{2^n}.$$

Equivalently: flip all bits of x and add 1.

- Range: $-2^{n-1} \leq x \leq 2^{n-1} - 1$. For $n = 8$: -128 to $+127$.
- The **sign bit** is the MSB: 0 for non-negative, 1 for negative.
- Adding a number to its two's complement gives all zeros — the "self-cancelling" property that makes subtraction free.

(P&H, Sec. 3.2, p.188; Hamacher Ch. 2) [3, 1]

Worked Example: $+7$ and -3 in 8-bit Two's Complement

Represent both

$+7 = 0000\,0111_2$. -3: flip $0000\,0011 \to 1111\,1100$, add 1 $\Rightarrow 1111\,1101_2$.

Check by adding

$0000\,0111 + 1111\,1101 = 1\,0000\,0100$. Discard the carry-out (9th bit)
$\Rightarrow 0000\,0100 = +4$. $7 + (-3) = 4$.

The thread, started

This is the thread's first full example: negative numbers are ordinary bits, and **addition needs no special subtraction hardware** — just discard the carry-out.

Socratic Check: Negating Without a Calculator

Question

-5 in 8-bit two's complement is $1111\,1011$. Without computing anything, what is $-(-5)$, i.e. the two's complement of $1111\,1011$? What do you expect, and why?

- Flip $1111\,1011 \to 0000\,0100$, add 1 $\Rightarrow 0000\,0101 = +5$.
- Negating twice returns the original — two's complement is an **involution**: $-(-x) = x$.
- This self-inverse property is why negation is easy to implement and verify.

Two's complement makes negatives look like ordinary bits. But arithmetic can still overflow the range.

What Is Overflow?

Definition

Overflow occurs when the true result of an arithmetic operation cannot be represented in the format's range.

- Unsigned: carry out of the MSB is overflow.
- Two's complement: overflow $\neq$ carry out. A carry out of the MSB does *not* always mean overflow (recall $7 + (-3)$ above).
- The correct condition: overflow occurs when the sign of the result is *wrong* — i.e. adding two positives gives a negative, or two negatives give a positive.

(P&H, Sec. 3.2, p.188) [3]

Worked Example: Overflow vs. Carry

8-bit two's complement (range -128 to $+127$)

- `+100 + +50`: $0110\,0100 + 0011\,0010 = 1001\,0110$, which reads as -106 — wrong sign, so **overflow**.
- `+127 + 1`: $0111\,1111 + 1 = 1000\,0000 = -128$ — again wrong sign, overflow.
- `-1 + (-1)`: $1111\,1111 + 1111\,1111 = 1\,1111\,1110$; discard carry $\Rightarrow 1111\,1110 = -2$ — correct.

Rule of thumb

Overflow happens exactly when the two operands share a sign bit and the result's sign bit differs. Carry-out alone is *not* the test.

Fixed-Point Representation

Fractional numbers with a fixed binary point

A fixed-point number has an agreed position for the "binary point":

$$b_{k-1} \ldots b_0 . b_{-1} b_{-2} \ldots b_{-m} \quad \text{value} = \sum_i b_i 2^i.$$

- E.g. `Q8.8`: 8 integer bits, 8 fraction bits — each fractional bit is $2^{-1}, 2^{-2}, \ldots$.
- $0.5 = 0.1_2$; $0.25 = 0.01_2$; $0.75 = 0.11_2$.
- Simple and fast (add/sub work on the same integer hardware), but the fixed point limits range and precision — which is why Week 3 moves to floating point.

(Hamacher Ch. 2; Mano Ch. 1) [1, 2]

Worked Example: A Fixed-Point Addition

Add $3.5 + 0.75$ in Q8.8

$3.5 = 0000\,0011.1000\,0000$; $0.75 = 0000\,0000.1100\,0000$.

Add bitwise

$0000\,0011.1000\,0000 + 0000\,0000.1100\,0000 = 0000\,0100.0100\,0000$. Read off: 4.25.
Correct: $3.5 + 0.75 = 4.25$.

The integer ALU adds fixed-point values unchanged — the programmer just agrees where the point is.

Socratic Check: When Does Fixed Point Fail?

Question

What happens if you try to store 0.1 in binary — does it terminate? What consequence does that have for fixed-point arithmetic?

- 0.1 in binary is a **repeating** fraction ($0.000110011\ldots_2$) — it does not terminate.
- Fixed point must **truncate**, so 0.1 is stored approximately — repeated arithmetic accumulates error.
- This is the fundamental limitation that floating point (Week 3) manages but does not eliminate.

We can represent negatives and fractions. Now: the hardware that actually adds.

Motivation: From Addition to a Circuit

Socratic question

A human adds two numbers one column at a time, carrying. What is the smallest circuit that adds two *single bits* plus a carry-in — and how do we chain them into a multi-bit adder?

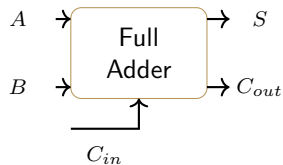
- **Half adder** (HA): two inputs A, B; outputs sum S and carry C.
- **Full adder** (FA): three inputs A, B, C_{in}; outputs sum S and carry C_{out}.
- Chain n full adders: each FA's C_{out} feeds the next C_{in}.

Definition: Half Adder and Full Adder

Boolean equations

`Half adder:` $S = A \oplus B, \quad C = A \cdot B.$

`Full adder:` $S = A \oplus B \oplus C_{in}, \quad C_{out} = A \cdot B + C_{in} \cdot (A \oplus B).$

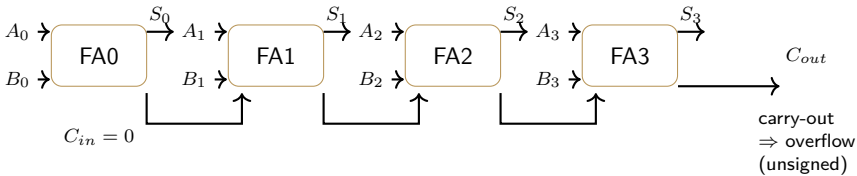


Worked Example: Full Adder Truth Table

A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Count the 1s in (A, B, C_{in}): if odd, $S = 1$; if ≥ 2, $C_{out} = 1$.

Ripple-Carry Adder: Chaining the Full Adders



The carry **ripples** from the least-significant to the most-significant bit — hence the name. The final C_{out} is the unsigned overflow flag.

Worked Example: Adding $7 + (-3)$ in Hardware

The thread, closed

$+7 = 0111$, $-3 = 1101$ (4-bit two's complement). Add with a 4-bit ripple-carry adder, $C_{in} = 0$:

Column by column

$0111 + 1101$: bit 0: $1 + 1 = 0$ carry 1; bit 1: $1 + 0 + 1 = 0$ carry 1; bit 2: $1 + 1 + 1 = 1$ carry 1; bit 3: $0 + 1 + 1 = 0$ carry 1. Result $0100 = +4$, carry-out $= 1$ (discarded). $7 + (-3) = 4$, no overflow — signs match the result.

One adder, same circuit, handles both signed and unsigned — the interpretation is the programmer's choice.

Socratic Check: Adder and Subtraction, One Circuit

Question

The ALU has only an adder. To compute $A - B$, what single extra step is needed — and what does it imply for the cost of subtraction hardware?

- Subtract B's two's complement: $A - B = A + (-B)$.
- $-B$ = flip B + add 1 — implementable with a MUX (choose B or $\bar{B}$) plus a forced carry-in of 1.
- So subtraction is **free**: one adder + a MUX, no separate subtractor needed. (P&H, Sec. 3.2, p.188) [3]

We have the representation and the hardware. Now: put it together.

The Thread, Closed

What we built today

- **Number systems**: positional notation; conversion by repeated division; hex/octal as bit grouping.
- **Two's complement**: $-x \equiv 2^n - x$; range -2^{n-1} to $2^{n-1} - 1$; negation = flip + 1.
- **Overflow**: wrong sign bit after addition of two same-signed operands — not the same as carry-out.
- **Fixed point**: agreed binary point; integer hardware works unchanged; fractions repeat in binary.
- **Adder**: HA $\rightarrow$ FA $\rightarrow$ ripple-carry; $A - B = A + (-B)$ is free.

The running thread

$+7 + (-3)$: represent both in two's complement, add with a ripple-carry adder, discard the carry-out, check the sign — 4, no overflow.

Bridge to Week 3

What's still missing

Fixed point handles fractions but with fixed range/precision. Real numbers span enormous ranges (from 10^{-300} to 10^{300} in science) — one fixed point cannot cover them.

Next week

Floating point: the IEEE 754 representation — sign, exponent, mantissa — trading a little precision for a huge range. Week 3 builds the format, the normalization, and the arithmetic.

Summary

- **Number systems**: value $= \sum d_i r^i$; convert by repeated division; hex/octal group bits.
- **Two's complement**: $-2^{n-1} \leq x \leq 2^{n-1} - 1$; negate by flip $+ 1$; addition needs no special subtraction hardware.
- **Overflow**: carry-out $\neq$ overflow; wrong result sign is the test.
- **Fixed point**: an agreed binary point; simple but limited range and precision.
- **Adder**: HA/FA equations; ripple-carry chain; $A - B = A + (-B)$.

References


V. Carl Hamacher, Zvonko G. Vranesic, and Safwat G. Zaky.
Computer Organization.
McGraw-Hill, 5th edition, 2002.


M. Morris Mano.
Computer System Architecture.
Prentice-Hall, 3rd edition, 1993.


David A. Patterson and John L. Hennessy.
Computer Organization and Design: The Hardware/Software Interface.
Morgan Kaufmann, arm edition (5th) edition, 2017.